Practically, we ask for the minimal amount of tests to cover all (coverable) lines. Note that such reductions may not be safe in terms of bug-finding ability. Tests that execute the same lines may show different behavior.

Weighted set cover problem

The *weighted set cover problem* (WSCP) is a modification to the SCP. In addition to the definition of the SCP, we attribute weights (or "costs") to each subset of U in F. Let $\mathbb{W} : F \mapsto \mathbb{N}^+$ be a function that attributes a weight w_i to each $S_i \in F$, then the WSCP asks to find a subset of F whose union equals the universe and the sum of all associated weights is minimal:

$$\text{Find } R \subseteq \{1, 2, \ldots, m\} \tag{3.62}$$

$$\text{such that } \bigcup_{i \in R} S_i = U \tag{3.63}$$

$$\text{and } \sum_{i \in R} w_i \leq \sum_{i \in X} w_i \tag{3.64}$$

$$\forall X \subseteq \{1, 2, \ldots, m\} \text{ where } \bigcup_{i \in X} S_i = U. \tag{3.65}$$

WSCP: line coverage and execution time

We can relate the WSCP to line coverage and tests execution time by using the same transformation as for the SCP before and additionally associate weights in terms of execution time (say, in minutes) to each test and therefore to each set of covered lines.

SCP and WSCP are NP-hard

The SCP and the WSCP are NP-hard. Therefore, we do not expect to find an algorithm with a polynomial time complexity that guarantees to find an optimal solution for all inputs. For example, simply enumerating all combinations is not feasible in practice. For SAP HANA, the problem size typically consists of 4 million elements (lines) within 2 000 subsets. Enumerating all solutions is equivalent to setting bits in a binary number with 2000 bits. Each bit represents whether a specific subset is included or not. Therefore, enumerating all combinations would require calculating $2^{2000} \approx 10^{602}$ possible inputs. Furthermore, for each input, we must iterate over 4 million elements. Given these points, it is not feasible to enumerate all combinations for this problem size. However, even for large problem instances, we can apply an approach consisting of the following steps:

- Apply logical reductions with polynomial time complexity to potentially simplify a given problem instance, i.e., reduce the cardinality of F and the cardinalities of the elements in F.
- Find an optimal solution by a full search for small problem sizes.
- Find a near-optimal solution with a heuristic, i.e., a polynomial time greedy algorithm that achieves an approximation ratio of $H(n)$ where n is the cardinality of U. In this case, $H(n)$ is the n-th harmonic number, i.e., $H(n) = \sum_{i=1}^{n} 1/i \leq ln(n) + 1$.

We discuss these steps in the following sections. There is also a wide range of research proposing different heuristics [30, 79, 161].

3.3.5 Logical Reductions for the SCP

We can apply several logical reductions for the SCP in a repeated approach, namely *duplicate removal*, *subset removal* and *must-have item identification*. Each of these reductions has an execution time that depends on the current